Proxy

Proxy · Lazy-load heavy image assets · Build with Waqar

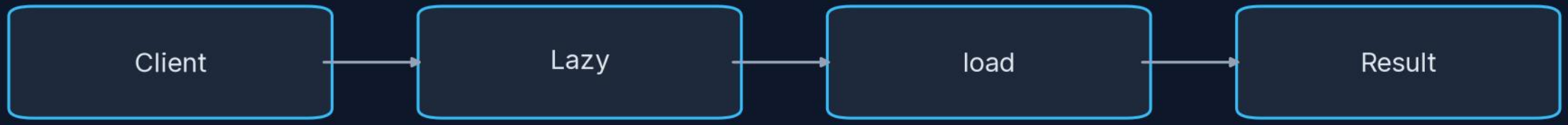
Today's Hook

- Problem: Lazy-load heavy image assets
- Pattern: Proxy — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Lazy-load heavy image assets
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Proxy: Lazy-load heavy image assets

Real-World Scenario

- Running example: Lazy-load heavy image assets
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.structural.proxy.problem;

import java.util.*;

public final class NaiveImage {
  public NaiveImage(String path){ System.out.println("LOAD "+path); }
  public String display(){ return "shown"; }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.structural.proxy.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;

interface Image { String display(); boolean loaded(); }
final class RealImage implements Image {
  private final String path; RealImage(String path){ this.path=path; System.out.println("LOAD "+path); }
  public String display(){ return "shown:"+path; } public boolean loaded(){ return true; }
}
final class LazyImageProxy implements Image {
  private final String path; private RealImage real;
  LazyImageProxy(String path){ this.path=path; }
  public String display(){ if(real==null) real=new RealImage(path); return real.display(); }
  public boolean loaded(){ return real!=null; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.structural.proxy.problem;

import java.util.*;
import java.util.concurrent.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Proxy: lazy-load heavy image ===");
    System.out.println("--- naive (loads immediately) ---");
var n=new NaiveImage("/big.png"); System.out.println(n.display());
System.out.println("--- proxy ---");
Image p=new LazyImageProxy("/big.png");
System.out.println("loadedBefore="+p.loaded());
System.out.println(p.display());
System.out.println("loadedAfter="+p.loaded());
  }
}
```

Walkthrough Output

```
=== Proxy: lazy-load heavy image ===
--- naive (loads immediately) ---
LOAD /big.png
shown
--- proxy ---
loadedBefore=false
LOAD /big.png
shown:/big.png
loadedAfter=true
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Proxy.

Related Patterns

- Contrast Proxy with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Proxy → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.